PilotSwarm Eval Harness: Proposal

---

1. Summary

This document describes the PilotSwarm Eval Harness: a new `packages/eval` module that provides an SDK-native evaluation system for PilotSwarm agents. It covers **22 evaluation dimensions** across **100+ scenarios**, and it integrates with Langfuse for observability, prompt management, and trace visualization.

The proposal is organized around two questions:

1. **What does the eval harness cover?** (Section 3: every dimension and scenario)
2. **How does it work?** (Section 4: the execution model, grading pipeline, and tooling)

---

2. Architecture at a Glance

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          EVAL HARNESS CLI / SDK                          │
│  CLI: pilotswarm eval run --suite=smoke --model=claude-sonnet-4          │
│  SDK: import { EvalRunner, EvalSuite } from "pilotswarm-eval"            │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  ┌──────────────────┐  ┌──────────────────┐  ┌──────────────────┐       │
│  │ DATASET MANAGER  │  │ TASK RUNNER      │  │ GRADER PIPELINE  │       │
│  │                  │  │                  │  │                  │       │
│  │ Versioned golden │  │ PilotSwarmClient │  │ Layer 1: Code    │       │
│  │ datasets (JSON)  │  │ and Worker       │  │ (deterministic   │       │
│  │                  │  │                  │  │  assertions)     │       │
│  │ Synthetic gen    │  │ Multi-trial      │  │                  │       │
│  │ via LLM          │  │ execution        │  │ Layer 2: LLM     │       │
│  │                  │  │                  │  │ (Claude judge    │       │
│  │ Trace-to-dataset │  │ OTel trace       │  │  with rubrics)   │       │
│  │ feedback loop    │  │ capture          │  │                  │       │
│  │                  │  │                  │  │ Layer 3: Stats   │       │
│  │                  │  │ Fault injection  │  │ (multi-trial     │       │
│  │                  │  │ middleware       │  │  aggregation)    │       │
│  └──────────────────┘  └──────────────────┘  └──────────────────┘       │
│                                                                          │
│  ┌──────────────────┐  ┌──────────────────┐  ┌──────────────────┐       │
│  │ LANGFUSE         │  │ OTel TRACING     │  │ COST TRACKING    │       │
│  │ (self-hosted)    │  │                  │  │                  │       │
│  │                  │  │ OpenTelemetry    │  │ Token counting   │       │
│  │ Prompt registry  │  │ GenAI semantic   │  │ per session and  │       │
│  │ with versioning  │  │ conventions      │  │ per agent        │       │
│  │                  │  │                  │  │                  │       │
│  │ Trace backend    │  │ Langfuse as      │  │ Dollar cost per  │       │
│  │ and eval UI      │  │ primary backend  │  │ eval task        │       │
│  │                  │  │                  │  │                  │       │
│  │ A/B prompt       │  │ Grafana via      │  │ Budget ceilings  │       │
│  │ testing          │  │ OTel Collector   │  │ with alerts      │       │
│  │                  │  │ (future)         │  │                  │       │
```

```
│  └──────────────────┘  └──────────────────┘  └──────────────────┘          │
│                                                                             │
├─────────────────────────────────────────────────────────────────────────────┤
│                        CHAOS AND FAULT INJECTION                            │
│  Worker crash mid-turn | Tool timeout and failure | CMS partition           │
│  Timer drift | Sub-agent crash | Concurrent write races                     │
├─────────────────────────────────────────────────────────────────────────────┤
│               HUMAN-IN-THE-LOOP REVIEW AND CALIBRATION                      │
│  Reviewer dashboard | Rubric calibration | Score override and annotation    │
│  Gold-standard injection | Inter-rater reliability tracking                 │
├─────────────────────────────────────────────────────────────────────────────┤
│                       REPORTING AND CI/CD GATES                             │
│  Tier 1: code-graded on every PR (<2 min, $0 cost)                          │
│  Tier 2: smoke eval on merge to main (<15 min, ~$15 cost)                   │
│  Tier 3: full suite nightly (<60 min, ~$150 cost)                           │
│  Regression detection via Mann-Whitney U test against baselines             │
└─────────────────────────────────────────────────────────────────────────────┘
```

---

3. What the Eval Harness Covers

The eval harness evaluates 22 distinct dimensions. Each dimension targets a specific aspect of PilotSwarm behavior, and each contains multiple scenarios with explicit grading criteria.

These dimensions were identified through a systematic audit of the PilotSwarm SDK source code, all 50+ existing integration tests, the orchestration layer, documentation, contracts, and architectural proposals.

Dimension Summary

#	Dimension	Scenarios	Priority	Phase
1	Session Lifecycle	4	P0	1
2	Orchestration Health	4	P0	1
3	Multi-Agent Behavior	5	P0	1
4	Tool Use	4	P0	1
5	Cost and Performance	4	P0	1
6	Security and Guardrails	5	P0	1
7	Timer and Scheduling	6	P0	1
8	Long-Running Agent Liveness	6	P0	1
9	Lossy Handoff and Recovery	5	P0	1
10	Session Policies and System Agents	6	P0	1
11	Dehydration and Blob Store	4	P1	1
12	Prompt Quality	3	P1	2
13	Facts and Knowledge	6	P1	2

14	Inline Control Tools	5	P1	2
15	Context Window and Compaction	5	P1	2
16	Orchestration Versioning	3	P1	2
17	Prompt Layering	5	P1	2
18	Model Provider and Selection	5	P1	2
19	Human-in-the-Loop Evaluation	5	P1	2
20	Management and Admin	5	P2	3
21	Sweeper and Resource Manager	4	P2	3
22	Child Update Batching	3	P2	3
	Total	**~102**		

Dimension Details

Each dimension below lists its scenarios along with the grading approach. "Code Grader" means a deterministic programmatic assertion (zero cost, instant). "LLM Judge" means a rubric-based evaluation by a strong judge model (Claude Sonnet with chain-of-thought scoring).

1. Session Lifecycle

Scenario	Code Grader	LLM Judge
Create session, send message, receive response	Session appears in CMS with correct state	Response is coherent and on-topic
Multi-turn conversation with memory	Prior context referenced in later turns	Conversation coherence across turns
Session resume after disconnect	Session resumes from last known state	No topic drift or memory loss
Session with custom system prompt	System prompt reflected in agent behavior	Instruction-following quality

2. Orchestration Health

Scenario	Code Grader	LLM Judge
Deterministic replay produces consistent results	No nondeterminism errors in orchestration logs	
Durable timer fires within acceptable tolerance	Timer actual vs. expected delta is less than 5%	
Activity completes and records result	Activity recorded in orchestration history	

ContinueAsNew preserves all carried state	All CAN fields (timers, prompts, cron, context) survive	

3. Multi-Agent Behavior

Scenario	Code Grader	LLM Judge
Spawn a sub-agent with correct metadata	Parent-child link in CMS, correct agentId, title, model	
Nested agents at depth 2+	Full agent tree visible in CMS	Delegation quality
Cascade cancellation cleans up agent tree	All descendants cancelled when parent cancelled	
Result propagation from child to parent	Parent receives and acts on child result	Task coherence
Multiple sub-agents spawned in parallel	All children created, no race conditions in CMS	Parallel delegation quality

4. Tool Use

Scenario	Code Grader	LLM Judge
Agent selects the correct tool for a task	Tool call recorded with expected tool name	Selection reasoning quality
Tool parameters are accurate	Parameter values match expected schema	
Tool error is handled gracefully	Agent does not crash or loop on tool failure	Recovery reasoning quality
Multi-tool sequence in correct order	Tool call sequence matches expected execution plan	Execution plan quality

5. Cost and Performance

Scenario	Code Grader	LLM Judge
Token usage per session stays within budget	Total tokens less than configured ceiling	
Turn latency stays within acceptable range	P95 latency less than threshold (e.g., 10s)	
Dollar cost per eval task is tracked	Cost recorded in trace metadata	
Model comparison across same scenarios	Cost and quality scores compared per model	Quality-per-dollar ranking

6. Security and Guardrails

Scenario	Code Grader	LLM Judge

Prompt injection in user message detected	Guardrail fires, injection blocked	Refusal quality and helpfulness
Prompt injection in tool output detected	Untrusted content wrapped correctly	
Authority claim injection blocked	Agent does not follow injected "system" instructions	
Content wrapping applied to sub-agent responses	Wrapper markers present around untrusted content	
Agent refusal is polite and informative		Refusal tone and clarity

7. Timer and Scheduling

Scenario	Code Grader	LLM Judge
Wait interrupt: user message wakes a waiting session	Session resumes, timer cancelled	Response coherence
Wait affinity: `preserveWorkerAffinity: true`	Same worker handles the resume	
Cron set and cancel	Cron fires at correct intervals, cancel stops it	
Cron and wait coexistence	Both timers remain active simultaneously	
Idle timer fires correctly	Session transitions to idle state after timeout	
Input-grace timer provides correct window	Grace period elapses before input_required timeout	

8. Long-Running Agent Liveness

This dimension specifically targets the scenario where a sub-agent is spawned with a recurring schedule (such as a cron job, a periodic check, or a timed wake cycle) and must reliably wake up, remember its context, perform the correct work, and go back to sleep at the right time. This is one of the most failure-prone patterns in durable agent systems, because it combines timer reliability, state persistence, and behavioral correctness across long time horizons.

Scenario	Code Grader	LLM Judge
Wake reliability: Agent spawned with a cron interval wakes up on every scheduled tick. It does not "sleep forever" or skip ticks.	Count of actual wake events matches expected count over N intervals. Timer deltas are within tolerance.	
State retention across wake cycles: Agent remembers what it learned or	Facts or session state from cycle N are accessible in cycle N+1. Key-value pairs persist	Agent demonstrates

decided in previous wake cycles. Accumulated knowledge is not lost.	across ContinueAsNew boundaries.	coherent memory of prior work
Correct task execution on each wake: Agent performs the correct job on each wake cycle, not just an empty or confused response. It calls the expected tools and produces meaningful output.	Expected tool calls present in each cycle. Output is non-empty and structurally valid.	Task quality and relevance per cycle
Interval accuracy and consistency: Cron or wait intervals fire at the configured cadence. There is no progressive drift, no doubling, and no irregular gaps.	Standard deviation of inter-wake intervals is within 10% of expected period. No missed intervals.	
Graceful degradation on missed wake: If a wake is missed (due to worker crash, CMS partition, or timer drift), the agent detects the gap and compensates appropriately. It does not silently skip work or double-execute.	Recovery event logged. No duplicate work. No silent data loss.	Recovery reasoning quality
Long session lifespan without resource leak: Agent running for many cycles does not accumulate unbounded state, memory leaks, or context window overflow. Compaction and CAN keep the session healthy.	Context usage stays within bounds. CAN triggers at expected intervals. No OOM or timeout.	

9. Lossy Handoff and Recovery

Scenario	Code Grader	LLM Judge
Closed CopilotSession triggers retry and `session.lossy_handoff` event	Event recorded, UI text rendered	
Lost warm session recovers without infinite retry	Session recovers within bounded attempts	Behavioral continuity
Concurrent session isolation (6+ parallel sessions)	No cross-session state corruption	
Failed runtime status overrides stale CMS state	CMS shows `Failed` status, not stale `active`	
Terminal child rejects new messages	Send to completed child returns error	

10. Session Policies and System Agents

Scenario	Code Grader	LLM Judge
Session creation policy: allowlist enforcement	Disallowed agents rejected	

Session creation policy: generic rejection	Non-agent sessions rejected if policy requires it	
System session delete protection	System sessions survive `deleteSession()` calls	
`maxSessionsPerRuntime` cap enforcement	Session creation fails at cap	
System agent auto-start on worker boot	System agents running after `worker.start()`	
System agent idempotent restart	Re-start does not create duplicates (stable UUIDs)	

11. Dehydration and Blob Store

Scenario	Code Grader	LLM Judge
Dehydrate then hydrate roundtrip fidelity	State hash matches pre-dehydration	
Checkpoint persistence (filesystem or S3)	Checkpoint exists and is readable	
Missing or corrupt archive handling	Graceful fallback, no crash	
Artifact persistence and retrieval	`write_artifact` followed by download succeeds	

12. Prompt Quality

Scenario	Code Grader	LLM Judge
A/B prompt comparison	Both variants produce valid sessions	Which variant performs better on rubric
Prompt regression detection	Score delta against baseline	Statistical significance of change
Instruction-following fidelity		Agent follows explicit instructions in prompt

13. Facts and Knowledge

Scenario	Code Grader	LLM Judge
Store, read, delete fact roundtrip	Fact persisted, retrieved, removed	
Namespace ACLs (`skills/`, `asks/`, `intake/`)	Writes rejected outside allowed namespaces	
Lineage access: child reads parent's facts	Ancestor facts visible to child	

Shared vs. session-scoped facts isolation	Session facts do not leak to sibling sessions	
Facts cleanup on session delete	Descendant facts removed	
Intake to Facts Manager promotion	Intake fact promoted to shared, original cleaned	

14. Inline Control Tools

Scenario	Code Grader	LLM Judge
`spawn_agent` executes inline (no turn abort)	Parent turn continues after spawn	Task continuity
`message_agent` executes inline	Parent receives acknowledgment, continues reasoning	
`wait_for_agents` suspends turn	Turn pauses until children complete	
Multiple inline spawns in one turn	All children created, parent reasoning continues	Task delegation quality
Explicit turn boundary (`complete_agent`)	Turn ends only when agent explicitly completes	

15. Context Window and Compaction

Scenario	Code Grader	LLM Judge
Context usage events propagate to session info	Usage numbers present in `getInfo()`	
Compaction triggers near context limit	Compaction snapshot recorded	
Post-compaction behavior correctness	Agent continues coherently after compaction	Memory retention quality
ContinueAsNew carries state correctly	All state fields survive CAN checkpoint	
Warm resume after CAN	Queued actions (spawns, prompts) survive	Behavioral continuity

16. Orchestration Versioning

Scenario	Code Grader	LLM Judge
Session created under v1.0.30 resumes under v1.0.36	No nondeterminism errors	Behavioral consistency
Timer state migrates across versions	Active timers fire correctly post-upgrade	

CAN state format migration	All fields carry across version boundary	

17. Prompt Layering

Scenario	Code Grader	LLM Judge
Composition order: framework, app default, active agent, runtime	Layers applied in correct order	
Framework layer override	Framework base prompt overrides applied correctly	
Replace vs. append mode	Replace mode discards prior layers; append mode stacks	
Active agent prompt applied at session creation	`.agent.md` content visible in composed prompt	
Langfuse hot-swap prompt takes effect	New session uses updated Langfuse prompt, not stale `.agent.md`	

18. Model Provider and Selection

Scenario	Code Grader	LLM Judge
Qualified model name resolves correctly (e.g., `anthropic/claude-sonnet-4`)	Correct provider and model in CMS	
Bare model name fallback (e.g., `claude-sonnet-4`)	Resolves to first matching provider	
Invalid provider filtered out	Error returned, session not created	
Mid-session model switch	Model change reflected in subsequent turns	
Model comparison: same scenario, different models	Both models produce valid sessions	Quality and cost comparison

19. Human-in-the-Loop Evaluation

Automated grading (code assertions and LLM judges) is powerful but insufficient on its own. LLM judges can be confidently wrong, and some behavioral qualities (such as tone, trust, and domain appropriateness) are difficult to capture with rubrics. Human-in-the-loop evaluation provides a calibration layer that keeps the automated grading pipeline honest.

Scenario	How It Works
Reviewer dashboard for sampled sessions	A random sample of eval traces (configurable, such as 10% of nightly runs) is surfaced in the Langfuse dashboard for human review. Reviewers score each session on a rubric and can override or annotate any automated score.

Gold-standard injection for judge calibration	Pre-scored "gold-standard" sessions (where the correct score is known) are mixed into the eval pipeline at a configured rate. If the LLM judge scores a gold-standard session incorrectly, an alert fires, flagging judge drift. This catches cases where the judge becomes confidently wrong.
Inter-rater reliability tracking	When multiple human reviewers score the same session, inter-rater agreement (Cohen's kappa) is computed and tracked over time. Low agreement on a dimension signals that the rubric needs clarification or the dimension is too subjective for the current grading criteria.
Score override with annotation	Any automated score (code grader or LLM judge) can be overridden by a human reviewer with a required annotation explaining the reason. Overrides feed back into the training data for future rubric refinement.
Eval-gated prompt promotion	Before a prompt version is promoted from `dev` to `staging` or from `staging` to `prod`, a human reviewer must approve a summary report of the eval results. This prevents automated promotion of prompts that pass on metrics but fail on subjective quality.

20. Management and Admin

Scenario	Code Grader	LLM Judge
Session rename via management client	CMS title updated	
Session cancel via management client	Session transitions to cancelled state	
Command routing (send_command)	Command delivered and processed	
Execution history retrieval	Full turn history returned	
Session dump for debugging	Dump contains all state, events, and metadata	

21. Sweeper and Resource Manager

Scenario	Code Grader	LLM Judge
Zombie session detection	Stale sessions identified and marked	
System session protection	System sessions excluded from sweeper cleanup	
Dry-run mode safety	Dry-run lists targets without acting	
Resource cleanup after threshold	Sessions cleaned after exceeding age or idle limits	

22. Child Update Batching

Scenario	Code Grader	LLM Judge
Digest window batches child updates	Updates within window merged into single digest	

Latest-wins deduplication	Multiple updates from same child collapsed	
Merged digest injected into parent prompt	Parent sees batched summary, not individual updates	Digest quality

---

4. How the Eval Harness Works

This section describes the execution model, grading pipeline, tooling, and integration mechanisms.

4.1 Eval Runner: SDK-Native TypeScript

The eval runner lives in `packages/eval` and is written in TypeScript. It uses `PilotSwarmClient` and `PilotSwarmWorker` directly, giving it first-class access to CMS session state, orchestration status, event streams, and duroxide runtime internals.

Interfaces:

- **CLI:** `pilotswarm eval run --suite=smoke --model=claude-sonnet-4`
- **Programmatic API:** `import { EvalRunner, EvalSuite } from "pilotswarm-eval"`
- **MCP server:** The existing PilotSwarm MCP server tools (`create_session` , `send_and_wait` , `spawn_agent` , etc.) serve as a secondary driver, enabling external LLM agents to run eval scenarios through the MCP protocol.

Reuse: The runner reuses the existing `withClient()` test infrastructure and assertion helpers from the current 50+ integration test suite. This means every eval scenario starts from a proven foundation.

4.2 Eval Task Format

Each eval scenario is defined as a JSON task. Here is an example:

```
{
  "id": "liveness-wake-reliability",
  "dimension": "long-running-agent-liveness",
  "priority": "P0",
  "description": "Agent spawned with 30s cron wakes on every tick for 5 cycles",
  "setup": {
    "agent": "cron-monitor",
    "model": "claude-sonnet-4",
    "systemMessage": "You are a monitoring agent. On each wake cycle, check the
status of your assigned service and report any changes.",
    "cronInterval": "30s",
    "expectedCycles": 5
  },
  "steps": [
    { "action": "spawn_agent", "params": { "agentId": "cron-monitor", "cron": "*/30
* * * * *" } },
    { "action": "wait", "duration": "180s" },
    { "action": "collect_events", "filter": "session.cron_wake" }
  ],
  "graders": {
    "code": [
      { "check": "event_count", "field": "cron_wake", "expected": 5, "tolerance": 1
```

```
},
      { "check": "interval_consistency", "expectedMs": 30000, "maxDeviationPct": 10
}
    ],
    "llm": {
      "rubric": "Does the agent perform meaningful work on each wake cycle? Rate 1-
5.",
      "passThreshold": 3
    }
  },
  "trials": 3,
  "timeoutSeconds": 240,
  "tags": ["liveness", "cron", "P0"]
}
```

4.3 Grading Pipeline: Swiss Cheese Layering

Every eval task passes through three grading layers. This follows Anthropic's "Swiss Cheese" model: no single grader catches everything, but stacking them provides comprehensive coverage.

Layer 1: Code Graders (deterministic, instant, zero cost) Programmatic assertions that check CMS session state, verify tool calls were made with correct parameters, confirm orchestration health (no nondeterminism errors), validate sub-agent parent-child metadata, and enforce token budget limits. Code graders catch approximately 70% of failures with no LLM API cost.

Layer 2: LLM-as-Judge (behavioral, nuanced, moderate cost) A strong model (Claude Sonnet) evaluates agent behavior using structured rubrics with chain-of-thought reasoning. The judge scores task completion quality, trajectory efficiency, tool selection reasoning, and recovery behavior after faults. To mitigate known biases (position bias, verbosity bias, self-preference), the judge model is always from a different model family than the agent under test.

Layer 3: Statistical Aggregation (handles non-determinism) Each eval task runs 3 to 5 trials to account for LLM non-determinism. Results are aggregated using pass@k scoring (success in any of k trials) and bootstrap confidence intervals. Regression detection uses Mann-Whitney U tests against historical baselines, alerting when degradation exceeds 10%.

4.4 Chaos and Fault Injection

A fault injection middleware wraps PilotSwarm's duroxide activity boundary, enabling systematic chaos testing of all durable execution paths. Faults are injected at the activity layer (not deep inside CopilotSession), so they test the full recovery pipeline including orchestration replay.

The chaos taxonomy follows MAS-FIRE (published 2026), which defines 15 fault categories for multi-agent systems spanning cognitive errors, coordination errors, and infrastructure failures.

Pre-built chaos scenarios:

Scenario	Faults Injected	What We Grade
Worker crash mid-turn	Kill the worker process during an active LLM call	Session resumes on a new worker with no data loss

Tool timeout cascade	All registered tools time out for 30 seconds	Agent retries or gracefully degrades without looping
CMS database partition	CMS becomes unavailable for 10 seconds	Orchestration buffers operations and recovers without crashing
Durable timer drift	Timers fire 50% later than expected	Cron schedules still execute in the correct order
Sub-agent worker crash	The worker running a child agent crashes	Parent detects the failure and either re-spawns or reports it
Concurrent write race	Multiple workers attempt to write the same session	No state corruption occurs in CMS

4.5 Observability: OTel Tracing with Langfuse

The eval harness instruments PilotSwarm with OpenTelemetry GenAI semantic conventions and sends traces to Langfuse as the primary backend.

Trace hierarchy:

```
eval.run (eval suite execution)
  └── eval.task (single eval scenario)
      └── eval.trial (one attempt of the task)
          └── pilotswarm.session (session lifecycle)
              ├── pilotswarm.turn (orchestration turn)
              │   ├── gen_ai.chat (LLM call, with model, tokens, latency)
              │   ├── gen_ai.execute_tool (tool invocation, with name and result)
              │   └── pilotswarm.activity (duroxide activity execution)
              ├── pilotswarm.spawn_agent (sub-agent creation)
              ├── pilotswarm.timer (durable timer, with expected vs. actual duration)
              └── pilotswarm.cms_transition (session state change)
```

Because the OTel GenAI conventions are still marked experimental (as of April 2026), we build a thin abstraction layer (`traceAgent()` , `traceTurn()` , `traceTool()`) that maps to OTel attributes. If attribute names change, only the abstraction layer needs updating. We also use Langfuse's native SDK for critical tracing paths as a stability fallback, converging to OTel-only once conventions reach stable status.

Grafana dashboards can be added later by inserting an OTel Collector as a fanout layer, which requires zero changes to the instrumentation code.

4.6 Prompt Management: Langfuse Integration

Langfuse (self-hosted) serves as the centralized prompt registry, trace backend, and evaluation dashboard.

How prompt management works:

- The existing `.agent.md` files remain as committed defaults in version control.
- Langfuse acts as the hot-swap iteration layer. At session creation, the SDK checks Langfuse for a prompt (by label: `prod` , `staging` , or `dev`). If a matching prompt exists, it is used. If not, the SDK falls back to the `.agent.md` file.
- Prompt engineers can update an agent prompt in Langfuse and immediately test it with a new session, without redeploying any code.

- Prompts are versioned and promoted through environments (dev to staging to prod). Promotion from staging to prod requires eval gate approval (automated scores must pass, and optionally a human reviewer must approve).

Operational note: Self-hosted Langfuse requires ClickHouse (trace storage), PostgreSQL (metadata), and Redis (caching). We recommend using Langfuse Cloud for dev/staging environments and self-hosting only for production.

4.7 Human-in-the-Loop Calibration

The human review layer is not a separate workflow; it is woven into the automated pipeline at specific checkpoints:

1. **Sampling:** A configurable percentage of eval traces (for example, 10% of nightly runs) is flagged for human review in the Langfuse dashboard.
2. **Gold-standard injection:** Pre-scored sessions with known-correct grades are injected at a configured rate. If the LLM judge misgrades a gold-standard session, an alert fires.
3. **Override flow:** Human reviewers can override any automated score with a required annotation. Overrides feed back into rubric refinement.
4. **Promotion gate:** Before a prompt is promoted to production, a human reviewer approves the eval summary report.

4.8 Datasets

Eval tasks are grouped into versioned golden datasets stored in `packages/eval/datasets/`:

- **Seeded from existing tests:** The initial dataset (v1) is derived from the 50+ existing integration tests, converted to the eval task JSON format.
- **Synthetic generation:** An LLM generates edge-case scenarios for dimensions that need broader coverage. Synthetic tasks are reviewed before inclusion.
- **Trace-to-dataset feedback loop:** Production traces that reveal interesting failures can be captured and converted into new eval tasks, continuously expanding coverage.

4.9 CI/CD Integration: Three-Tier Eval Gates (OPTIONAL)

Tier	Trigger	Duration	Cost	What Runs
Tier 1	Every pull request	Less than 2 minutes	$0	Code graders only. Schema validation, type checks, CMS state assertions, orchestration health verification. No LLM calls.
Tier 2	Merge to main branch	Less than 15 minutes	~$15	Smoke eval suite: 10 to 15 critical scenarios with single-trial LLM runs. Covers session lifecycle, tool calling, sub-agent spawn, and basic chaos.
Tier 3	Nightly schedule	Less than 60 minutes	~$150	Full eval suite: all 100+ scenarios, 3 to 5 trials each, LLM-as-judge scoring, chaos variants, statistical regression detection against historical baselines.

---

5. Implementation Phases

Phase 1: Foundation (P0 dimensions, ~50 scenarios)

Covers: Session Lifecycle, Orchestration Health, Multi-Agent Behavior, Tool Use, Cost and Performance, Security and Guardrails, Timer and Scheduling, Long-Running Agent Liveness, Lossy Handoff and Recovery, Session Policies and System Agents, Dehydration and Blob Store.

Delivers: `packages/eval` scaffolding, EvalRunner class, code graders, LLM judge integration, fault injection middleware, golden dataset v1, CLI interface, Langfuse tracing integration.

Phase 2: Intelligence (P1 dimensions, ~40 scenarios)

Covers: Prompt Quality, Facts and Knowledge, Inline Control Tools, Context Window and Compaction, Orchestration Versioning, Prompt Layering, Model Provider and Selection, Human-in-the-Loop Evaluation.

Delivers: Langfuse prompt registry integration, A/B testing pipeline, gold-standard injection, reviewer dashboard, synthetic dataset generation, statistical regression detection.

Phase 3: Production (P2 dimensions and operational, ~15 scenarios)

Covers: Management and Admin, Sweeper and Resource Manager, Child Update Batching.

Delivers: Full CI/CD integration (if adopted), Grafana dashboards via OTel Collector, trace-to-dataset feedback loop, production monitoring with alerting.